

R2S-Eval: Robot Evaluation with Real-to-Sim Calibration via Vision-Language Models

Yidi Wang^{1,2,*}, Feixiang Ruan^{1,3,*}, Ruoku Chen⁴, Jie Yin¹, Yang Yu², Mengdi Xu⁴, Kaifeng Zhang^{1,†}

Abstract—Evaluating robot manipulation policies is becoming increasingly important as generalist models, particularly vision-language-action (VLA) models, are deployed on physical robots. However, conventional real-world evaluation remains labor-intensive, unstable, and insufficiently informative. It requires repeated hardware trials, manual scene resets, and continuous operator monitoring, may produce different policy rankings across repeated evaluations, and primarily relies on success-rate metrics that provide limited information about execution quality. In contrast, humans assess robot performance by observing and comparing complete behaviors rather than relying solely on binary success outcomes. To this end, we propose R2S-Eval, an evaluation pipeline that combines real-to-sim calibration with vision-language model (VLM) preference evaluation. The real-to-sim component efficiently generates rollout videos in a simulator calibrated to the real-world evaluation setting, thereby reducing the need for repeated hardware trials. The VLM evaluator assesses the execution quality of rollout videos and produces pairwise preferences, which are subsequently aggregated into policy rankings. We further introduce a protocol to assess whether the proposed evaluation pipeline yields validated policy conclusions while mitigating the key challenges of conventional real-world evaluation. Experiments in both simulation and real-world settings demonstrate that R2S-Eval produces reliable and stable policy conclusions, achieves agreement with human preferences, substantially reduces repeated hardware-operation effort, and reveals behavior-quality differences that are not captured by binary success labels. In general, R2S-Eval advances robot evaluation from manual success counting toward automated, statistically stable, and quality-aware evaluation of robot behavior. Project page: <https://r2s-eval.github.io>.

I. INTRODUCTION

Recent progress in generalist manipulation policies, such as vision-language-action (VLA) models, is bringing physical robots closer to solving real-world tasks [1], [2], [3], [4]. In this context, evaluation is essential for measuring how well different policies perform in real settings. A well-designed evaluation does more than report performance. It reveals strengths and failure modes, guides data collection and training, and helps identify how to improve robot policies.

Current policy evaluation typically follows a protocol of task-specific fine-tuning and hardware deployment. Candidate policies are fine-tuned on downstream task data and deployed in a manually configured real-world environment, where operators conduct repeated trials and record task successes. Policies are ranked by their success rates, and the resulting ranking is treated as a proxy for the real-world manipulation capability of the underlying generalist models [4], [5], [6].

Although effective, this hardware evaluation loop is labor-intensive and unstable. Each trial requires scene setup, robot execution, object reset, and hardware monitoring, leading to a growing manual effort. The resulting rankings are difficult to repeat consistently, as small changes in object placement, contact, perception, or robot state can alter rollout outcomes [6]. Moreover, the success rate is too coarse and provides limited information on execution quality. Policies with similar success rates may differ substantially in motion quality, corrective behavior, or progress before failure.

These limitations motivate a more informative view of robot policy evaluation, one that is closer to how humans assess robot behavior. When watching a rollout, humans consider the complete execution rather than only whether the final goal is reached. They can distinguish two successful rollouts that differ in control quality, or two failed rollouts that make different amounts of progress toward the goal. This observation suggests evaluating policies from rollout videos and estimating policy quality through preferences over observed executions rather than only success counts [7].

However, video-based preference evaluation is not by itself practical. Collecting rollout videos on hardware still requires repeated manual effort, while video comparison introduces additional annotation effort. A practical video-based preference evaluation must therefore address two problems: how to efficiently obtain behavior videos consistent with the real world, and how to judge behavior quality automatically in a way that reflects human preferences.

To this end, we propose R2S-Eval, illustrated in Fig. 1, an evaluation pipeline that combines real-to-sim calibration with vision-language model (VLM) preference evaluation. The real-to-sim component constructs a simulation calibrated to the real world. Candidate policies are adapted with simulated data and deployed in closed loop in simulation to produce rollout videos efficiently. A VLM then assesses the execution quality of rollout videos and compares sampled video pairs to produce preferences that are aggregated into policy rankings. We further introduce a validation protocol that examines whether these resulting conclusions reflect model capability, stabilize with increasing trials, agree with human preferences, reduce repeated hardware-operation effort, and reveal behavior differences beyond success counts.

In summary, the main contributions of this work are:

- We formulate manipulation policy evaluation as preference estimation over rollout behaviors, producing policy rankings that reflect execution quality rather than binary success counts.
- We introduce R2S-Eval, an evaluation pipeline that

*Equal contribution; †Corresponding author; ¹Sharpa; ²Nanjing University; ³Tongji University; ⁴Tsinghua University

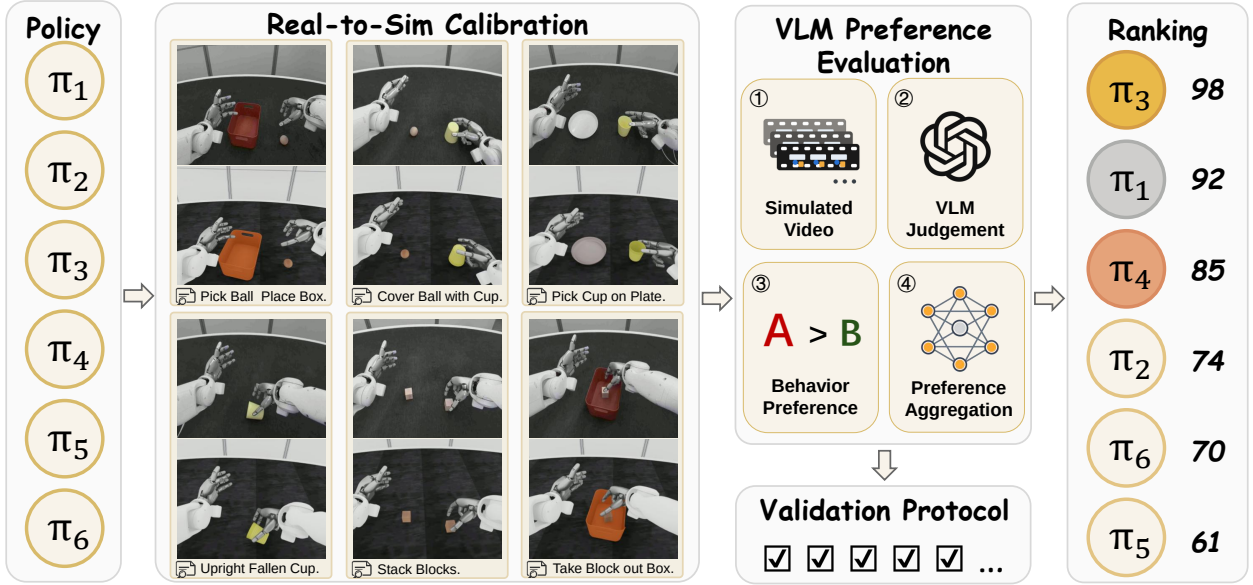


Fig. 1. Overview of the R2S-Eval pipeline for ranking robot policies using real-to-sim calibration and VLM-based preference evaluation.

obtains behavior videos from a calibrated real-to-sim simulation and ranks policies using VLM-generated preferences.

- We provide a multi-axis validation protocol for policy evaluation methods covering ranking reliability and stability, human agreement, hardware manual effort, and behavioral informativeness beyond success.

II. RELATED WORK

Real World and Simulation Policy Evaluation. Robot policy evaluation spans standardized simulation benchmarks, automated real-world testing, and proxy evaluation through simulators or learned world models. Recent benchmarks assess language-conditioned manipulation, long-horizon reasoning, robustness, and cross-task generalization [8], [9], [10], [11], [12], [5], [13], [14]. Real-world systems automate success detection, reset, and policy scheduling, or distribute blinded comparisons across sites [15], [16], while proxy methods study whether simulated performance predicts hardware results [6], [17]. However, these approaches still emphasize success rates, scalar scores, or aggregate rank correlations, leaving behavioral differences and ranking stability insufficiently characterized. Large-scale embodied benchmarks such as BEHAVIOR-1K [18] further expand evaluation toward long-horizon household activities, but evaluation is still primarily based on task completion metrics.

Real-to-Sim Environment Construction. Real-to-sim methods reconstruct physical environments for policy training and evaluation using geometric reconstruction, neural rendering, 3D Gaussian Splatting, and physical simulation [19], [20], [21], [22], [23], [24]. Related work transfers video-reconstructed hand-object interactions into simulation to recover tactile supervision [25]. Recent frameworks further use reconstructed environments for sim-real comparison, progress estimation, and preference aggregation [26], [27].

Nevertheless, contact, detection, and control mismatches remain difficult to reproduce, and evaluation is still dominated by task results or aggregate agreement.

VLM-Based Robot Behavior Evaluation. VLMs have been used for success detection, task-progress estimation, failure diagnosis, subgoal verification, and execution-quality assessment [28], [29], [30]. Although these methods capture process-level differences beyond binary success, most assign absolute labels or scores to individual trajectories. Existing policy ranking approaches also rely heavily on expert annotations or learned evaluators [30]. Recent LLM-as-a-Judge studies demonstrate the effectiveness of pairwise preference evaluation for ranking complex model outputs [31], [32]. Aggregating repeated pairwise VLM judgments into statistically grounded and validated policy rankings remains underexplored.

III. R2S-EVAL PIPELINE

In this section, we present the R2S-Eval evaluation pipeline. Sec. III-A formulates the evaluation of policies as ranking candidate policies based on preferences over rollout behaviors. Sec. III-B describes real-to-sim calibration, collecting closed-loop rollout videos in a real-world calibrated simulation. Sec. III-C introduces the VLM preference evaluation, where a VLM judges videos and generates preferences that are aggregated into policy rankings. Sec. III-D presents a validation protocol to examine the evaluation pipeline.

A. Problem Formulation

We consider the problem of evaluating a set of candidate manipulation policies $\Pi = \{\pi_1, \pi_2, \dots, \pi_N\}$ under a target evaluation distribution $\mathcal{D}$, which specifies the tasks, initializations, and environment settings. A rollout of policy π_i is denoted by $\tau_i^k = (o_0, a_0, o_1, a_1, \dots, o_T)$, where o_t and a_t are observation and action at time t . We denote the video of a rollout by $v_i^k = \phi(\tau_i^k)$. The goal is to produce a ranking

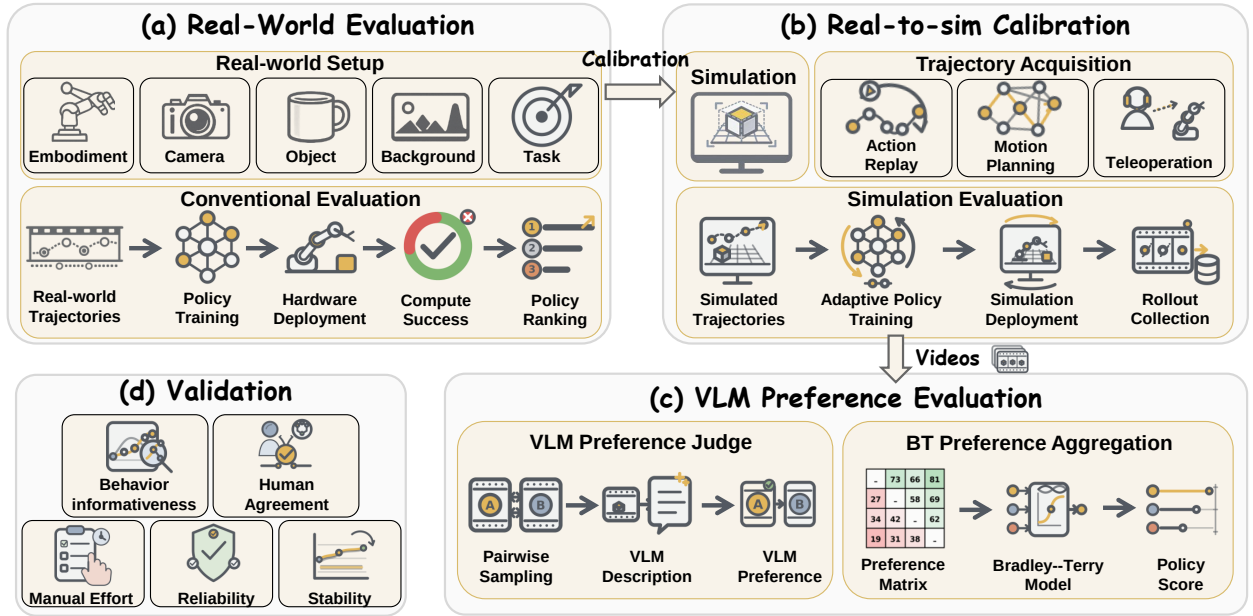


Fig. 2. Detailed workflow of the R2S-Eval pipeline. To address the limitations of (a) real-world robot evaluation, (b) the calibrated real-to-sim simulator generates rollout videos for (c) VLM-based preference evaluation. (d) The full pipeline is validated.

r over Π , where higher-ranked policies are judged to have better manipulation capabilities under $\mathcal{D}$.

Conventional evaluation, shown in Fig. 2 (a), ranks policies according to task success. Let $c(\tau) \in \{0, 1\}$ indicate whether a rollout is successful. The ranking is obtained by sorting the empirical success rate:

$$\begin{aligned} \hat{s}_i^{\text{succ}} &= \frac{1}{K_i} \sum_{k=1}^{K_i} c(\tau_i^k), \\ r^{\text{succ}} &= \text{Rank}(\hat{s}_1^{\text{succ}}, \dots, \hat{s}_N^{\text{succ}}), \end{aligned} \quad (1)$$

where $\text{Rank}(\cdot)$ orders policies. Since $c(\tau)$ reduces each rollout to a binary outcome, r^{succ} ignores the behavior differences among rollouts with the same success label.

We instead formulate the policy evaluation as preference estimation over rollout behaviors. Let $\mathcal{V}_i = \{v_i^1, \dots, v_i^{K_i}\}$ denote the rollout videos of π_i . For a sampled pair (v_i^a, v_j^b) , a preference judge outputs $y_{ij}^{ab} \in \{i \succ j, j \succ i\}$. Let $\mathcal{C}_{ij}$ denote the comparisons between π_i and π_j . The observed preferences are summarized by a matrix $M \in \mathbb{N}^{N \times N}$,

$$M_{ij} = \sum_{(a,b) \in \mathcal{C}_{ij}} \mathbf{1}[y_{ij}^{ab} = i \succ j] \quad (2)$$

where M_{ij} counts how often rollouts from π_i are preferred over rollouts from π_j .

To infer a ranking from M , we introduce a statistical preference model. Let θ_i denote the latent quality score of the policy π_i and $q_{ij}(\theta) = \Pr_{\theta}(\pi_i \succ \pi_j)$ be the probability that π_i is preferred over π_j . For independent binary comparisons, the log-likelihood of M is

$$\ell(\theta; M) = \sum_{i < j} [M_{ij} \log q_{ij}(\theta) + M_{ji} \log q_{ji}(\theta)], \quad (3)$$

where $q_{ji}(\theta) = 1 - q_{ij}(\theta)$. Then the scores and ranking are

$$\begin{aligned} \hat{\theta} &= \arg \max_{\theta} \ell(\theta; M), \\ r^{\text{pref}} &= \text{Rank}(\hat{\theta}_1, \dots, \hat{\theta}_N). \end{aligned} \quad (4)$$

Thus, policy evaluation shifts from sorting success counts to estimating a ranking from preferences over rollout behaviors.

B. Real-to-Sim Calibration

Sec. III-A assumes access to the rollout videos $\mathcal{V}_i$ for each candidate policy. Directly collecting these videos inherits the limitations of conventional real-world evaluation, which is labor-intensive and unstable. To address the video acquisition problem, we use real-to-sim calibration, shown in Fig. 2 (b), as an efficient source of behavior videos.

Let $\mathcal{D}_R$ denote the real-world evaluation distribution and $\mathcal{D}_S$ its calibrated simulation counterpart. The goal is not to build a perfectly photorealistic digital twin, but to calibrate the factors that determine policy behavior, including robots, tasks, cameras, and initializations. The simulated robot matches the real geometry, kinematics, joint limits, and control interface. The simulated tasks match their real-world counterparts, and objects are reconstructed as 3D assets and placed according to pose estimates. The camera viewpoints are calibrated to match the real geometry.

We then obtain evaluation videos through three stages.

Adaptive policy training. Although calibration narrows the real-to-sim gap, residual differences in friction, compliance, and sensing noise remain unavoidable in contact-rich manipulation. Direct deployment in simulation may cause a policy to exhibit unstable behavior, making the collected rollouts reflect residual domain mismatch rather than policy differences of interest. We therefore acquire calibrated simulation trajectories through action replay, motion planning,

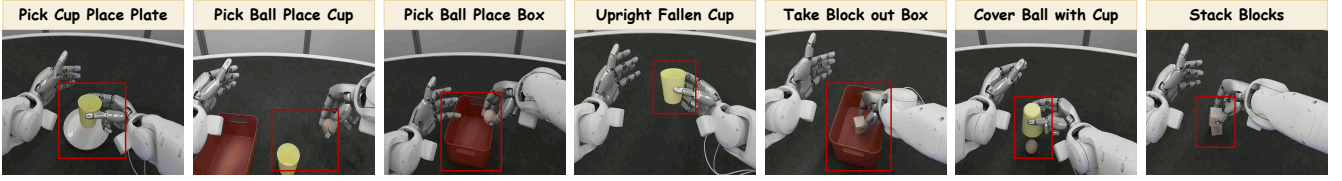


Fig. 3. Illustration of the real-world manipulation task suite in experiments.

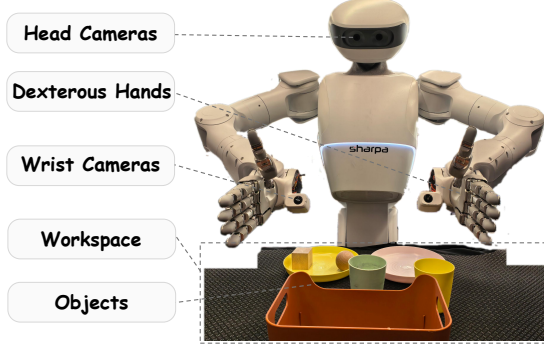


Fig. 4. SharpaNorth robot platform.

or teleoperation, and adapt candidate policies. Let π_i^R denote the policy evaluated in the real world and π_i^S its simulation-adapted counterpart derived from the same underlying candidate. R2S-Eval directly evaluates π_i^S and does not assume that it is identical to π_i^R . Instead, the comparative ranking induced by $\{\pi_i^S\}$ is used as a proxy for the hardware ranking of $\{\pi_i^R\}$. Sec. IV-A assesses this proxy through closed-loop task performance and ranking consistency.

Closed-loop simulation deployment. Each policy π_i^S is deployed in closed loop in the calibrated simulation. Evaluation videos are collected from the policy’s own executions.

Rollout video collection. For each π_i^S , we record

$$\mathcal{V}_i = \{v_i^1, \dots, v_i^{K_i}\}, \quad (5)$$

which serves as input to VLM preference evaluation. With calibrated simulation, R2S-Eval reduces the need for repeated hardware trials.

C. VLM Preference Evaluation

Given the video sets $\{\mathcal{V}_i\}_{i=1}^N$, VLM preference evaluation, shown in Fig. 2 (c), estimates the preference observations. Humans watch videos and compare the overall quality of execution, serving as a natural reference. However, this introduces substantial manual effort. We therefore use a vision-language model (VLM) as an automated video-based preference judge. The VLM is used not as a hand-designed task scorer but as a general visual judge in a way intended to match human preference. We evaluate this human agreement in Sec. IV-B. The module consists of 3 stages.

Pairwise video sampling. For each pair (π_i, π_j) , we sample

$$(v_i^a, v_j^b), \quad v_i^a \in \mathcal{V}_i, v_j^b \in \mathcal{V}_j. \quad (6)$$

Each comparison uses videos under the same task and initial configuration, and each task contributes the same number of

comparisons. Repeated comparisons over randomly sampled task configurations therefore estimate preference between policy rollout distributions without confounding policy quality with task frequency or condition difficulty.

Structured VLM judgment. Given a video v_i^a and its instruction x , the VLM gives a structured behavior description

$$d_i^a = G_{\text{desc}}(v_i^a, x). \quad (7)$$

The description covers motion smoothness, temporal continuity, task progress, and visually observable contact and effort control in the video. For a pair (v_i^a, v_j^b) , the two anonymized videos are randomly assigned to positions A or B . The VLM compares the corresponding descriptions under the same criteria and enforces a binary preference output

$$y_{ij}^{ab} = G_{\text{pref}}(d_i^a, d_j^b) \in \{i \succ j, j \succ i\}. \quad (8)$$

This two-step process grounds the pairwise judgment in explicit execution evidence rather than a success signal. The task-level win counts are pooled into a preference matrix M .

Bradley–Terry preference aggregation. To obtain the policy-level ranking from M , we instantiate the statistical preference model with the Bradley–Terry model [33]. The probability that π_i is preferred over π_j is modeled as

$$q_{ij}(\theta) = \Pr(\pi_i \succ \pi_j) = \frac{\exp(\theta_i)}{\exp(\theta_i) + \exp(\theta_j)} = \sigma(\theta_i - \theta_j). \quad (9)$$

We fit BT strength parameters $b_i > 0$ using the minorization-maximization algorithm [34] and impose $\sum_i b_i = 1$ for identifiability, with $\theta_i = \log b_i$. Using the likelihood objective, we estimate the policy scores $\hat{\theta}$ and rank the policies accordingly.

D. Validation Protocol

We complement the operational pipeline with a validation protocol in Fig. 2 (d) along five axes.

Reliability. A reliable evaluator should produce policy conclusions consistent with the target setting. For policy ranking, it should agree with a reference ranking that reflects the actual capabilities of the same set of policies.

Stability. As evaluation evidence accumulates, the estimated policy scores or rankings should converge and exhibit low variance across different trials and configurations.

Human agreement. The preference signal produced by the evaluator should agree with human judgments, indicating that its decisions reflect human assessment rather than an arbitrary model-specific scoring rule.

Human effort. Manual work may be required to obtain an evaluation result, including hardware execution, scene reset,

TABLE I
DATA STATISTICS ON REAL-WORLD TASKS.

Task	# Real Traj.	# Sim. Traj.	Replay Succ. (%)
Cover Ball with Cup	510	494	96.9
Pick Ball Place in Box	497	491	98.8
Pick Ball Place in Cup	509	491	96.5
Pick Cup Place on Plate	502	502	100.0
Stack Blocks	505	471	93.3
Take Block out Box	503	499	99.2
Upright Fallen Cup	499	497	99.6
Total / Avg.	3525	3445	97.8

TABLE II
REAL-SIM SUCCESS RATES ON 7 TASKS AND 6 VLAs.

Task	X-VLA	$\pi_{0.5}$	π_0	OpenVLA	Nora-Long	SmolVLA
Cover Ball Cup	60.0 / 61.0	55.0 / 57.0	55.0 / 60.5	5.0 / 3.5	5.0 / 3.5	5.0 / 1.5
Pick Ball Box	70.0 / 67.0	70.0 / 70.0	55.0 / 58.5	5.0 / 4.0	5.0 / 3.0	0.0 / 1.0
Pick Ball Cup	50.0 / 48.0	50.0 / 52.0	45.0 / 52.0	5.0 / 3.0	5.0 / 2.5	0.0 / 0.5
Pick Cup Plate	80.0 / 79.0	75.0 / 75.5	60.0 / 63.5	10.0 / 8.5	5.0 / 6.0	5.0 / 2.5
Stack Blocks	45.0 / 46.5	45.0 / 45.5	40.0 / 42.5	5.0 / 2.5	0.0 / 1.5	0.0 / 0.5
Take Block Box	65.0 / 67.5	65.0 / 65.0	55.0 / 58.0	5.0 / 4.5	5.0 / 2.5	0.0 / 1.0
Upright Cup	90.0 / 85.5	85.0 / 86.5	65.0 / 71.0	15.0 / 11.0	5.0 / 7.5	5.0 / 4.0
Total / Avg.	65.7 / 64.9	63.6 / 64.5	53.6 / 58.1	7.1 / 5.3	4.3 / 3.7	2.1 / 1.6

and monitoring. An accurate evaluator is still impractical if it requires substantial human intervention.

Behavioral informativeness. Behavioral information should be preserved. Binary success records only completion and discards other details. An informative evaluator should capture differences in progress, recovery, or control quality, especially given the same success or failure outcome.

IV. EXPERIMENTS

In this section, we conduct simulation and real-world experiments to answer the following questions: *Q1*. Does the calibrated simulation preserve hardware-consistent policy conclusions? *Q2*. Does R2S-Eval produce policy conclusions that reflect the actual performance of candidate policies? *Q3*. Can R2S-Eval mitigate the instability and manual cost of conventional real-world evaluation? *Q4*. Do VLM judgments agree with human preferences? *Q5*. Does preference-based evaluation capture execution-level information?

A. Real-to-Sim Calibration Results

This subsection answers *Q1* by introducing the real-world experimental setup and assessing the real-sim consistency.

Robot platform. All real-world experiments are conducted on SharpaNorth, a dual-arm humanoid manipulation platform shown in Fig. 4. The robot has two 7-DoF arms, each with a 22-DoF SharpaWave dexterous hand, as well as a 2-DoF neck, a 3-DoF upper body, and a 2-DoF waist. We keep the waist fixed, giving an active configuration of 63 DoFs. The observation system consists of two head-mounted RGB cameras for stereo workspace views and two wrist-mounted fish-eye cameras for close-range hand-object observations.

Task suite. We design seven tabletop tasks shown in Fig. 3. The manipulated objects are randomly initialized on the tabletop within a predefined range. Task objects, initialization ranges, and success conditions remain consistent between the real world and the corresponding simulation.

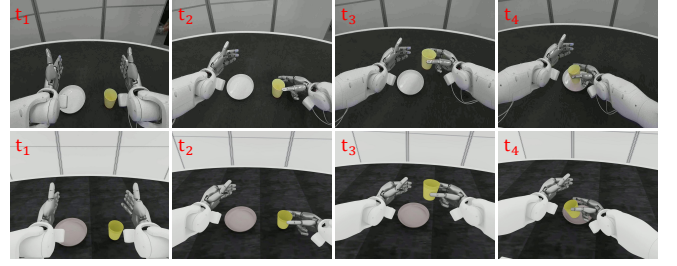


Fig. 5. A real-world trajectory (top) and its simulated action-replay counterpart (bottom) for the Pick Cup and Place on Plate task. Each row shows four key frames ordered from left to right.

VLA policies. We evaluate six candidate VLA policies: (1) π_0 [35], a flow-matching VLA built on a pretrained vision-language model; (2) $\pi_{0.5}$ [36], an open-world generalization variant of π_0 ; (3) **OpenVLA** [4], a 7B open source VLA pretrained on large-scale robot demonstrations; (4) **NORA-Long**, a long-action variant of NORA [37], which adopts Qwen2.5-VL-3B [38] and the FAST+ action tokenizer [39]; (5) **SmolVLA** [40], a lightweight VLA for efficient training and deployment; and (6) **X-VLA** [41], a VLA with specific soft prompts for cross-embodiment policy learning.

Real-world data collection. We collect real-world trajectories with a teleoperation system, composed of an upper-body exoskeleton, exoskeleton gloves, and a VR headset. The exoskeleton tracks the operator’s arm and hand motions and maps them to the robot action space, while the VR headset provides stereo visual feedback from the robot’s head-mounted cameras. During teleoperation, we record synchronized head-camera views, wrist-camera views, robot proprioception, and executed actions. We collect 3,525 trajectories in total, which are shown in Tab. I.

Simulated data acquisition. Following Sec. III-B, we build the simulation in NVIDIA Isaac Sim [42]. We replay the recorded action sequences in the calibrated simulator to acquire the simulated data. Action replay is sensitive to the initial poses of objects, as small errors can change contact timing and cause a real-world trajectory to fail in simulation. We therefore use an automatic replay loop. If a replay attempt fails, we resample a perturbation around the estimated initial object pose and execute the same action sequence again until the replay succeeds or the retry limit is reached. Fig. 5 compares representative real and simulated trajectories, and Tab. I reports the trajectory counts and the success rates of action replay. The real-world policy π_i^R and its simulation-side counterpart π_i^S are trained independently on real and simulated data, respectively, using identical task definitions, observation and action interfaces, and the model-specific training recipe.

Closed-loop real-sim consistency. We compare the results of π_i^R and π_i^S from independently executed closed-loop policy rollouts. Tab. II reports each real/sim success rate. We run 20 and 200 trials per task in the real world and in simulation, respectively. The two routes produce closely matched task-level success rates and the same overall policy

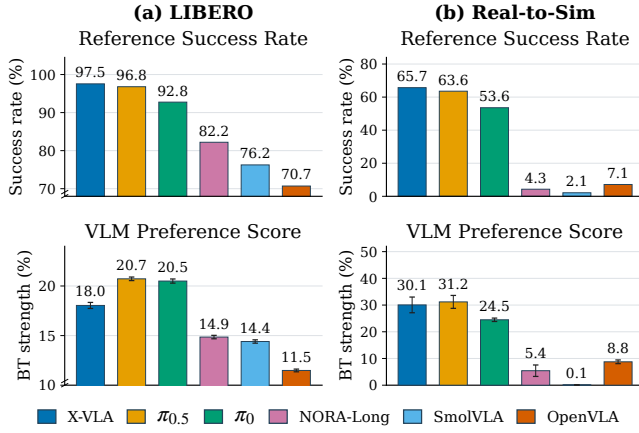


Fig. 6. Overall policy performance of LIBERO and Real-to-Sim experiments. The VLM preference score is averaged over all 8 VLMs.

ranking. Across all policy–task pairs, the mean absolute real–sim difference is 2.13 percentage points. The largest gap is 7.0 percentage points, indicating that calibration does not eliminate all discrepancies. In both domains, π_0 , $\pi_{0.5}$, and X-VLA clearly outperform the remaining policies, which may be limited by model capacity or action representation. Real and simulated rollouts also exhibit qualitatively similar failure modes, including near-static oscillations without progress and large-amplitude oscillations during motion. These results do not imply behavioral equivalence between π_i^R and π_i^S , but show that the calibrated simulation is sufficient to preserve comparative policy conclusions.

B. VLM Preference Evaluation Results

This subsection addresses *Q2*, the stability aspect of *Q3*, and *Q4* by evaluating the VLM preference evaluation results in simulated and calibrated real-to-sim settings.

Evaluation metrics. For *Q2*, we compare the results of the VLM with a reference performance. (1) **Spearman’s ρ** measures the agreement between the VLM ranking and the reference ranking. (2) **Pearson r** measures the correlation between VLM policy scores and reference performance values. (3) **MMRV** (Mean Maximum Rank Violation) [6] measures the severity of ranking inversions. For *Q3*, we recompute policy scores and rankings as more video comparisons are incorporated. (4) **BT CI** measures the average width of the 95% bootstrap confidence interval for standardized policy scores on a 0–100 scale. For *Q4*, we compare VLM preferences with human annotations. (5) **Human agreement rate** measures the percentage of pairs for which the VLM selects the same preferred video as human annotators.

Implementation details. (1) Reference performance is defined as the mean success rate on all tasks. (2) Bootstrap confidence intervals are estimated from 1,000 replicates. In each replicate, the number of comparisons is preserved, and the BT model is refitted. (3) Three human annotators view 200 video pairs and select the preferred ones under the same evaluation protocol as VLM. The sample pairs are approximately balanced across policies and outcome

TABLE III
LIBERO VLM PREFERENCE EVALUATION RESULTS.

VLM	$\rho \uparrow$	$r \uparrow$	MMRV $\downarrow$	BT CI $\downarrow$	Agr. rate (%) $\uparrow$
Qwen3-VL-4B	0.813	0.915	0.019	1.231	80.5
Qwen3-VL-8B	0.823	0.926	0.018	1.312	83.5
Qwen3-VL-32B	0.813	0.935	0.020	1.429	87.0
LLaVA-OV-1.5-4B	0.829	0.929	0.017	1.414	85.0
LLaVA-OV-1.5-8B	0.829	0.908	0.017	1.346	83.0
Gemma3-4B	0.829	0.903	0.017	1.285	78.0
Gemma3-12B	0.829	0.941	0.017	1.392	84.0
Gemma3-27B	0.823	0.939	0.018	1.285	82.0
Avg.	0.823	0.924	0.018	1.337	82.9

TABLE IV
REAL-TO-SIM VLM PREFERENCE EVALUATION RESULTS.

VLM	$\rho \uparrow$	$r \uparrow$	MMRV $\downarrow$	BT CI $\downarrow$	Agr. rate (%) $\uparrow$
Qwen3-VL-4B	0.943	0.957	0.007	1.712	90.0
Qwen3-VL-8B	0.943	0.984	0.007	1.750	91.5
Qwen3-VL-32B	1.000	0.973	0.000	1.771	92.0
LLaVA-OV-1.5-4B	0.943	0.989	0.006	1.827	88.5
LLaVA-OV-1.5-8B	1.000	0.984	0.000	1.807	90.0
Gemma3-4B	0.943	0.981	0.005	1.725	92.0
Gemma3-12B	0.943	0.968	0.009	1.722	93.0
Gemma3-27B	0.943	0.980	0.007	1.741	90.5
Avg.	0.957	0.978	0.005	1.757	91.9

categories. The majority vote defines the human reference preference. (4) The metrics are averaged across checkpoints ranging from 500 to 2,000 video comparisons per policy pair in increments of 100.

VLM evaluators. We evaluate eight VLMs. (1) **Qwen3-VL** [43] (4B, 8B, 32B), (2) **LLaVA-OV-1.5** [44] (4B, 8B), and (3) **Gemma 3** [45] (4B, 12B, 27B).

Simulation results. The LIBERO [5] results are summarized in Fig. 6 (a) and Tab. III. For each policy, we collect 2,000 rollout videos, 50 per task. Generally, the VLM ranking closely follows the success-rate ranking, with minor inversions among nearly tied policies. For example, $\pi_{0.5}$ receives a higher score than X-VLA despite a slightly lower success rate, reflecting its better execution quality. Across all VLMs, the inferred policy rankings are highly consistent, achieving an average Spearman’s ρ of 0.823, Pearson r of 0.924, and MMRV of 0.018. The narrow BT confidence intervals indicate robust ranking estimation with low uncertainty. The human agreement rate reaches 82.9%, demonstrating that VLM judgments agree with human assessments. All VLMs exhibit similar performance despite differences in architecture and model scale, suggesting that the proposed evaluation pipeline generalizes well across diverse VLM backbones.

Real-to-sim results. The calibrated real-to-sim results are shown in Fig. 6 (b) and Tab. IV. All policies use identical camera, rendering, and temporal-sampling settings, reducing the risk that policy-specific visual artifacts serve as preference cues. For each policy, we collect 1,400 rollout videos, 200 per task. The reference performance is computed in the real world. As policies exhibit larger performance differences on the real robot, the inferred rankings are more consistent than those on LIBERO. Notably, $\pi_{0.5}$ and X-VLA remain

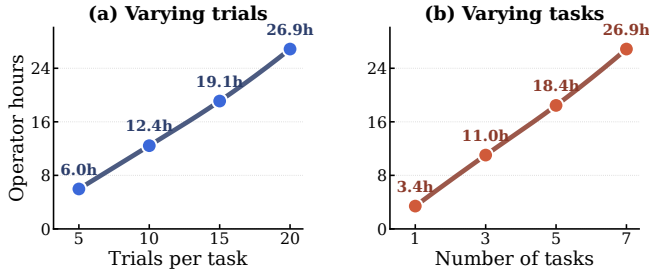


Fig. 7. Repeated hardware-operation effort.

nearly tied across all evaluated VLMs, with some VLMs marginally favoring X-VLA and others slightly preferring $\pi_{0.5}$. This behavior is consistent with their nearly identical real-world success rates, differing only by 2.1 percentage points. This indicates that the observed ranking differences reflect the small performance gap between the two policies.

Order-swap consistency. For each real-sim video pair, the A/B order is reversed and the evaluation is repeated. Across all VLMs, the preferred video remains unchanged for 91.5% of the cases. We observe a slight positional bias when comparing nearly tied video pairs, where VLMs tend to favor the video presented as A. To mitigate this, the two anonymized videos are randomly assigned to positions A and B, ensuring that any residual positional bias averages out over a large number of comparisons.

C. Hardware-operation Effort

This subsection addresses the manual effort aspect of *Q3* by quantifying the repeated operator time required for the conventional real-world evaluation. We count hardware execution, object initialization and reset, monitoring, and success recording but exclude one-time data preparation and computational costs. Human annotation is used only for validation and is not required during routine evaluation.

We keep the six candidate policies fixed and sum the measured operator time, while varying either the number of task or trials per task. As shown in Fig. 7, the repeated hardware-operation time increases approximately linearly, while R2S-Eval does not require additional real-world hardware effort.

D. Behavior Case Study

To address *Q5*, we examine rollout pairs with identical success labels. We select representative pairs in which both rollouts succeed or fail, and analyze whether VLM preference evaluation can distinguish their execution quality.

In Fig. 8 (a), both LIBERO rollouts are successful. The top grasps the object and places it smoothly in a single continuous execution, whereas the bottom fails its initial grasp and completes the task after a retry. In Fig. 8 (b), both real-world rollouts fail to complete the task. The top reaches into the box and attempts to manipulate the object before failing, while the bottom exhibits only small oscillatory motions without any progress. In both cases, the VLM prefers the first rollout for its higher execution quality. These examples demonstrate that VLM preference evaluation uses cues from

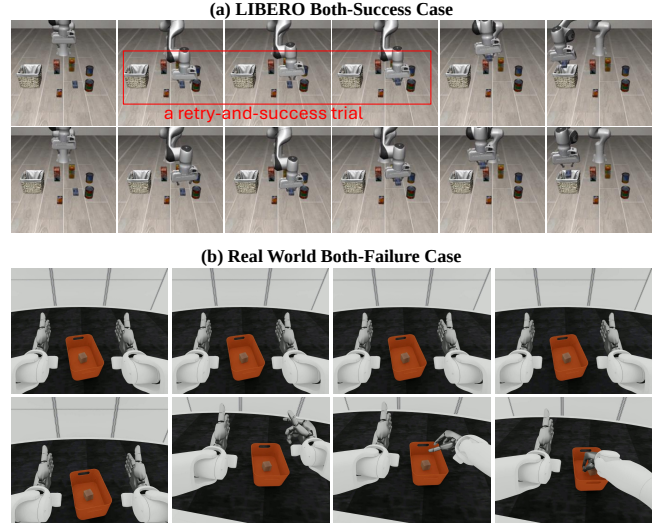


Fig. 8. Representative rollout pairs with identical binary outcomes. (a) Two successful rollouts for the LIBERO task “Pick up the cream cheese and place it in the basket.” (b) Two failed rollouts for the real-to-sim task “Take the Block out of the Box.” Each rollout proceeds from left to right.

the full execution process, including task progress, motion continuity, and control quality, and can therefore capture behavior differences invisible to binary success labels.

V. CONCLUSION

This work introduced R2S-Eval, an evaluation pipeline for robot manipulation policies based on preferences over rollout behaviors. It combines real-to-sim calibration for efficient video collection with VLM preference evaluation for policy ranking. Simulation and real-world experiments show that R2S-Eval produces reliable and stable policy conclusions, agrees with human judgments, reduces manual effort, and captures execution-quality differences beyond success counts. We believe that R2S-Eval represents a step toward a scalable and behavior-aware evaluation of robot manipulation policies. Future work will extend R2S-Eval to broader tasks and robots, while systematically analyzing real-to-sim discrepancies and constructing higher-fidelity simulations.

REFERENCES

- [1] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu *et al.*, “Rt-1: Robotics transformer for real-world control at scale,” 2022, arXiv:2212.06817.
- [2] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid *et al.*, “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” in *Conference on Robot Learning*. PMLR, 2023, pp. 2165–2183.
- [3] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, W. Huang, Y. Chebotar, P. Sermanet, D. Duckworth, S. Levine, V. Vanhoucke, K. Hausman, M. Toussaint, K. Greff, A. Zeng, I. Mordatch, and P. Florence, “PaLM-e: An embodied multimodal language model,” in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, Eds., vol. 202. PMLR, 23–29 Jul 2023, pp. 8469–8488. [Online]. Available: <https://proceedings.mlr.press/v202/driess23a.html>

- [4] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi *et al.*, “Openvla: An open-source vision-language-action model,” 2024, arXiv:2406.09246.
- [5] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “Libero: Benchmarking knowledge transfer for lifelong robot learning,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 44 776–44 791, 2023.
- [6] X. Li, K. Hsu, J. Gu, K. Pertsch, O. Mees, H. R. Walke, C. Fu, I. Lunawat, I. Sieh, S. Kirmani *et al.*, “Evaluating real-world robot manipulation policies in simulation,” 2024, arXiv:2405.05941.
- [7] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei, “Deep reinforcement learning from human preferences,” *Advances in neural information processing systems*, vol. 30, 2017.
- [8] Y. Wang, Z. Xian, F. Chen, T.-H. Wang, Y. Wang, K. Fragkiadaki, Z. Erickson, D. Held, and C. Gan, “Robogen: Towards unleashing infinite data for automated robot learning via generative simulation,” 2023, arXiv:2311.01455.
- [9] S. James, Z. Ma, D. R. Arrojo, and A. J. Davison, “Rlbench: The robot learning benchmark & learning environment,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 3019–3026, 2020.
- [10] Y. Zhu, J. Wong, A. Mandlekar, R. Martín-Martín, A. Joshi, K. Lin, A. Maddukuri, S. Nasiriany, and Y. Zhu, “robosuite: A modular simulation framework and benchmark for robot learning,” 2020, arXiv:2009.12293.
- [11] T. Yu, D. Quillen, Z. He, R. Julian, K. Hausman, C. Finn, and S. Levine, “Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning,” in *Conference on robot learning*. PMLR, 2020, pp. 1094–1100.
- [12] J. Gu, F. Xiang, X. Li, Z. Ling, X. Liu, T. Mu, Y. Tang, S. Tao, X. Wei, Y. Yao, X. Yuan, P. Xie, Z. Huang, R. Chen, and H. Su, “Maniskill2: A unified benchmark for generalizable manipulation skills,” in *International Conference on Learning Representations*, 2023.
- [13] S. Zhang, Z. Xu, P. Liu, X. Yu, Y. Li, Q. Gao, Z. Fei, Z. Yin, Z. Wu, Y.-G. Jiang, and X. Qiu, “Vlabench: A large-scale benchmark for language-conditioned robotics manipulation with long-horizon reasoning tasks,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2025, pp. 11 142–11 152.
- [14] T. Chen *et al.*, “Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation,” 2025, arXiv:2506.18088.
- [15] Z. Zhou, P. Atreya, Y. L. Tan, K. Pertsch, and S. Levine, “Autoeval: Autonomous evaluation of generalist robot manipulation policies in the real world,” 2025, arXiv:2503.24278.
- [16] P. Atreya *et al.*, “Roboarena: Distributed real-world evaluation of generalist robot policies,” 2025, arXiv:2506.18123.
- [17] Y. Li, Y. Zhu, J. Wen, C. Shen, and Y. Xu, “Worldval: World model as real-world robot policies evaluator,” 2025, arXiv:2505.19017.
- [18] C. Li, R. Zhang, J. Wong, C. Gokmen, S. Srivastava, R. Martín-Martín, C. Wang, G. Levine, W. Ai, B. Martinez *et al.*, “Behavior-1k: A human-centered, embodied ai benchmark with 1,000 everyday activities and realistic simulation,” 2024, arXiv:2403.09227.
- [19] S. Nasiriany, A. Maddukuri, L. Zhang, A. Parikh, A. Lo, A. Joshi, A. Mandlekar, and Y. Zhu, “Robocasa: Large-scale simulation of everyday tasks for generalist robots,” in *Robotics: Science and Systems (RSS)*, 2024.
- [20] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3d gaussian splatting for real-time radiance field rendering,” *ACM Transactions on Graphics*, vol. 42, no. 4, July 2023. [Online]. Available: <https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/>
- [21] M. Torne, A. Simeonov, Z. Li, A. Chan, T. Chen, A. Gupta, and P. Agrawal, “Reconciling reality through simulation: A real-to-sim-to-real approach for robust manipulation,” 2024, arXiv:2403.03949.
- [22] X. Li, J. Li, Z. Zhang, R. Zhang, F. Jia, T. Wang, H. Fan, K.-K. Tseng, and R. Wang, “Robogsim: A real2sim2real robotic gaussian splatting simulator,” 2024, arXiv:2411.11839.
- [23] X. Han, J. Yu, M. Liu, Y. Chen, X. Lyu, Y. Tian, B. Wang, W. Zhang, and J. Pang, “Re³sim: Generating high-fidelity simulation data via 3d-photorealistic real-to-sim for robotic manipulation,” 2026, arXiv:2502.08645.
- [24] K. Zhang *et al.*, “Real-to-sim robot policy evaluation with gaussian splatting simulation of soft-body interactions,” 2025, arXiv:2511.04665.
- [25] R. Chen, F. Ruan, L. Cao, Z. Wang, B. Xu, S. Tong, J. Liu, C. Zhang, M. Pei, W. Xing, K. Zhang, and M. Xu, “Dex-x: Learning visual-tactile dexterous manipulation from human videos with simulated interaction,” in *4th RSS Workshop on Dexterous Manipulation: Scalable Learning for Human-Level Skills*, 2026. [Online]. Available: <https://openreview.net/forum?id=fmyVZHjE1B>
- [26] A. Jain, M. Zhang, K. Arora, W. Chen, M. Torne, M. Z. Irshad, S. Zakharov, Y. Wang, S. Levine, C. Finn, W.-C. Ma, D. Shah, A. Gupta, and K. Pertsch, “Polaris: Scalable real-to-sim evaluations for generalist robot policies,” 2025, arXiv:2512.16881.
- [27] Y. Jangir, Y. Zhang, P.-C. Lo, K. Yamazaki, C. Zhang, K.-H. Tu, T.-W. Ke, L. Ke, Y. Bisk, and K. Fragkiadaki, “Robotarena ∞ : Scalable robot benchmarking via real-to-sim translation,” 2026, arXiv:2510.23571.
- [28] Y. Du, K. Konyushkova, M. Denil, A. Raju, J. Landon, F. Hill, N. de Freitas, and S. Cabi, “Vision-language models as success detectors,” 2023, arXiv:2303.07280.
- [29] J. Duan, W. Pumacay, N. Kumar, Y. R. Wang, S. Tian, W. Yuan, R. Krishna, D. Fox, A. Mandlekar, and Y. Guo, “Aha: A vision-language-model for detecting and reasoning over failures in robotic manipulation,” 2024, arXiv:2410.00371.
- [30] M. Liu, J. Sheng, P. Li, Z. Wang, T. Xu, T. Xu, and H. Liu, “Eval-actions: Fine-grained execution quality evaluation for robotic manipulation,” 2026, arXiv:2601.18723.
- [31] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing *et al.*, “Judging llm-as-a-judge with mt-bench and chatbot arena,” *Advances in neural information processing systems*, vol. 36, pp. 46 595–46 623, 2023.
- [32] W.-L. Chiang, L. Zheng, Y. Sheng, A. N. Angelopoulos, T. Li, D. Li, H. Zhang, B. Zhu, M. Jordan, J. E. Gonzalez *et al.*, “Chatbot arena: An open platform for evaluating llms by human preference,” 2024, arXiv:2403.04132.
- [33] R. A. Bradley and M. E. Terry, “Rank analysis of incomplete block designs: I. the method of paired comparisons,” *Biometrika*, vol. 39, no. 3/4, pp. 324–345, 1952.
- [34] D. R. Hunter, “MM algorithms for generalized Bradley-Terry models,” *The Annals of Statistics*, vol. 32, no. 1, pp. 384 – 406, 2004. [Online]. Available: <https://doi.org/10.1214/aos/1079120141>
- [35] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” 2024, arXiv:2410.24164.
- [36] Physical Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhahalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” 2025, arXiv:2504.16054.
- [37] C.-Y. Hung, Q. Sun, P. Hong, A. Zadeh, C. Li, U.-X. Tan, N. Majumder, and S. Poria, “Nora: A small open-sourced generalist vision language action model for embodied tasks,” 2025, arXiv:2504.19854.
- [38] S. Bai, K. Chen, X. Liu *et al.*, “Qwen2.5-vl technical report,” 2025, arXiv:2502.13923.
- [39] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong, O. Mees, C. Finn, and S. Levine, “Fast: Efficient action tokenization for vision-language-action models,” in *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025.
- [40] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti *et al.*, “Smolvla: A vision-language-action model for affordable and efficient robotics,” 2025, arXiv:2506.01844.
- [41] J. Zheng, J. Li, Z. Wang, D. Liu, X. Kang, Y. Feng, Y. Zheng, J. Zou, Y. Chen, J. Zeng *et al.*, “X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model,” 2025, arXiv:2510.10274.
- [42] NVIDIA, “Isaac sim,” Software, 2025, version 4.5.0. [Online]. Available: <https://github.com/isaac-sim/IsaacSim>
- [43] S. Bai *et al.*, “Qwen3-vl technical report,” 2025, arXiv:2511.21631.
- [44] X. An *et al.*, “Llava-onevision-1.5: Fully open framework for democratized multimodal training,” 2025, arXiv:2509.23661.
- [45] Gemma Team *et al.*, “Gemma 3 technical report,” 2025, arXiv:2503.19786.

Appendix

A. Real-World Setup

A.1. Hardware and Sensing

All real-world experiments are conducted on SharpaNorth¹, a dual-arm humanoid manipulation platform. The robot has two 7-DoF arms, each equipped with a 22-DoF SharpaWave² dexterous hand, as well as a 2-DoF neck, a 3-DoF upper body, and a 2-DoF waist. The observation system consists of 2 head-mounted RGB cameras (1920×1536) and 2 wrist-mounted fish-eye cameras (480×384). The cameras operate at 30 Hz. The action stream stores the commanded target joint angles, while the state stream stores the measured joint angles. Both action and state are recorded at 60 Hz. Timestamps are provided for every stream to synchronize the 60 Hz proprioception with the 30 Hz observations.

A.2. Action Space

Tab. V summarizes the unified action space. The unified action interface contains 65 joint targets. The two waist dimensions are retained in the vector for interface compatibility, but are held fixed during all experiments, leaving 63 actively controlled DoFs.

TABLE V
ACTION SPACE.

Index	Component	Joints	Dim.
0–6	Left arm	joint_1 ... joint_7	7
7–13	Right arm	joint_1 ... joint_7	7
<i>Left hand (22-DoF), indices 14–35</i>			
14–18	Thumb	CMC_FE, CMC_AA, MCP_FE, MCP_AA, IP	5
19–22	Index	MCP_FE, MCP_AA, PIP, DIP	4
23–26	Middle	MCP_FE, MCP_AA, PIP, DIP	4
27–30	Ring	MCP_FE, MCP_AA, PIP, DIP	4
31–35	Pinky	CMC, MCP_FE, MCP_AA, PIP, DIP	5
<i>Right hand (22-DoF), indices 36–57</i>			
36–40	Thumb	CMC_FE, CMC_AA, MCP_FE, MCP_AA, IP	5
41–44	Index	MCP_FE, MCP_AA, PIP, DIP	4
45–48	Middle	MCP_FE, MCP_AA, PIP, DIP	4
49–52	Ring	MCP_FE, MCP_AA, PIP, DIP	4
53–57	Pinky	CMC, MCP_FE, MCP_AA, PIP, DIP	5
58–64	Torso / neck	Upper body (3) + waist (2) + neck (2)	7
Total			65

A.3. Unified Policy Execution

To ensure a fair comparison, all evaluated policies are provided with the same camera streams and are executed through the same control interface, with identical control frequency, initialization, and termination criteria. Each model consumes the subset of available observations supported by its architecture, and the model-specific output is converted to the unified action representation before execution.

A.4. Objects and Assets

Real-world manipulation tasks involve a set of objects. To construct the calibrated simulation environment, simulation assets are created for all objects used in the experiments. For most manipulation objects (e.g., cups and plates), we reconstruct textured USD assets from the corresponding real objects through 3D scanning and texture mapping, where the object geometry is represented as a USD mesh and the appearance is preserved by texture maps. Simple scene elements and geometric objects, such as tables and walls, are manually modeled using primitive geometries.

A.5. Tasks

¹<https://www.sharpa.com/pages/north>

²<https://www.sharpa.com/pages/wave>

The seven real-world tasks and their success criteria are defined below. Unless otherwise specified, all movable objects are randomly initialized by sampling their positions (x,y) and yaw orientations within predefined regions on the tabletop while avoiding initial collisions. All closed-loop evaluation episodes use newly sampled initializations and are disjoint from the training trajectories. Task examples are shown in Fig. 9.

Cover the Ball with the Cup. The robot grasps the cup and places it upside down to cover the ball. The cup is initialized upside down. The task is considered successful if the ball is completely covered by the cup at the end of the episode.

Pick the Ball and Place it in the Box. The robot grasps the ball and places it inside the box. The task is considered successful if the ball is fully contained within the box.

Pick the Ball and Place it in the Cup. The robot grasps the ball and places it inside the cup. The task is considered successful if the ball is fully contained within the cup.

Pick the Cup and Place it on the Plate. The robot grasps the cup and places it upright on the plate. The task is considered successful if the cup is placed stably on the plate.

Stack the Two Blocks. The robot grasps one block and stacks it on top of the other block. The task is considered successful if one block is stacked stably on top of the other.

Take the Block out of the Box. The robot grasps the block from inside the box and places it outside the box. The task is considered successful if the block is completely removed from the box.

Upright the Fallen Cup. The robot grasps the fallen cup and restores it to an upright pose. The cup is initially placed in a fallen configuration. The task is considered successful if the cup remains upright at the end of the episode.

B. Real-to-Sim Calibration

B.1. Simulation Framework

Real-to-sim calibration is implemented using the NVIDIA Isaac Lab-Arena framework <https://github.com/isac-sim/IsaacLab-Arena>, an open-source extension to NVIDIA Isaac Lab for simplified task curation and robotic policy evaluation at scale.

B.2. Simulation Scene Construction

The simulation environment shares a common scene configuration for all tasks. The robot is instantiated from a calibrated USD articulation with the same kinematic structure as the real platform. Arms, lower body, and neck are controlled using implicit joint actuators, whereas dexterous hands use ideal PD actuators. Four RGB cameras are attached to the robot, including two head-mounted pinhole cameras (1920×1536) and two wrist-mounted fisheye cameras (480×384), which operate at approximately 30 Hz. Their mounting poses and projection models are configured according to the corresponding links and camera specifications of the physical robot.

The policy observation interface contains the four RGB streams, previous action, joint positions, and joint velocities. The images are rendered at their native camera resolutions and resized to 224×224 during model preprocessing. At the beginning of each episode, all joints are reset to their predefined initial configuration. The articulation enables collision handling and self-collision, and uses actuator gains and dynamics parameters stored in the calibrated robot USD asset.

The surrounding scene consists of a custom tabletop, a background wall, and dome lighting. The table and wall are loaded as rigid kinematic bodies and placed to reproduce the layout of the real-world workspace.

Task-specific manipulation objects are instantiated from their corresponding simulation assets and assigned their initial poses before setting up the environment. All tasks therefore share the same robot, workspace, cameras, lighting, and rendering configuration, differing only in the manipulation objects and their initial states. The tabletop height can be adjusted for individual task configurations when necessary.

The simulator exposes the same observation modalities and control interface as the real-world system for data collection and closed-loop evaluation.

Tab. VI summarizes the simulation parameters.

B.3. Task Implementation

Each simulation task is implemented to match the task specification of its real-world counterpart. Each simulated task uses the same language instruction, initialization protocol, and episode configuration as its real-world counterpart. Unless otherwise specified, movable objects are initialized by randomly sampling their positions and yaw orientations within predefined regions while avoiding initial collisions. Task completion is determined using rule-based success evaluators that are consistent with the real-world evaluation protocol. Examples of simulated tasks are shown in Fig. 10.

B.4. Initialization Estimation

TABLE VI
SIMULATION ENVIRONMENT CONFIGURATION.

Item	Value
Physics timestep	1/200 s (200 Hz)
Render timestep	1/100 s (100 Hz)
PhysX solver	TGS (<code>solver_type=1</code>)
Substeps	None (position-iteration based)
Articulation pos./vel. iters	8 / 0
Bounce threshold vel.	0.5 m/s
Friction offset thresh. / corr. dist.	0.04 / 0.025
Gravity	(0, 0, -9.81) m/s ²
Object static / dynamic friction	0.5 / 0.5 (Isaac Lab default)
Object restitution	0.0 (Isaac Lab default)
Robot contact offset / rest offset	0.002 / 0.0 m
Object contact / rest offset	PhysX default (not overridden)
Max depenetration vel. (robot)	1000 m/s
Self-collision (robot)	Enabled

We estimate the initial object poses in simulation through a three-stage procedure.

Segmentation of Real-World Objects. We perform open-vocabulary object segmentation using Grounding DINO for text-guided object detection and the Segment Anything Model (SAM) for mask refinement. The first RGB frame of each real-world trajectory is segmented to obtain object masks. Task-specific text prompts are used to detect all manipulation objects, from which the object centroids and principal orientations are extracted. These quantities provide the image-space observations required for initialization.

Real-Sim Pixel Alignment. The joint positions of the robot recorded at the first timestep are used to initialize the robot configuration in simulation. The object centroids are back-projected onto the tabletop using the calibrated camera intrinsics, extrinsics, and known tabletop height to recover their planar positions. The object yaw is estimated from the principal axis of each segmentation mask and transformed into the simulator world frame according to the camera pose. The initialized scene is then iteratively refined by minimizing the pixel reprojection error between the projected object centroids in the simulation and the corresponding centroids in the real image. At each iteration, the pixel reprojection error is converted into a correction in the tabletop coordinate frame through inverse projection, and the object positions are updated accordingly until convergence.

Task-Specific Error Calibration. For each task, a small set of trajectories, which are disjoint from all closed-loop evaluation rollouts, is replayed in simulation to calibrate task-specific initialization offsets. The replayed trajectories are compared with their real-world counterparts, and small systematic discrepancies caused by object geometry, perception ambiguity, or simulator dynamics are compensated by manually adjusting the initialization parameters. The resulting calibration offsets are fixed for the task and subsequently applied to all policy trajectories of the same task without further per-trajectory tuning.

B.5. Action Replay

We generate simulated trajectories by replaying the recorded joint-angle action sequences in the calibrated simulator. Replay outcomes are sensitive to small initialization errors because slight pose deviations may alter contact timing and cause a trajectory that succeeds in the real world to fail in simulation. We therefore employ an automatic replay procedure that searches within a small neighborhood of the estimated object poses. From the calibrated initialization, the recorded action sequence is executed and evaluated using the task-specific rule-based success criterion. If the replay fails, small pose perturbations are sampled around the estimated initialization and the trajectory is executed again. The procedure ends when a successful replay is obtained or the predefined retry limit is reached. Alg. 1 summarizes the automatic replay procedure. Fig. 15 presents additional examples of real-to-sim trajectory correspondence. We use $N = 8$ attempts. Translation offsets are sampled uniformly within $[-0.01, 0.01]\text{m} \times [-0.01, 0.01]\text{m}$, and yaw offsets within $[-\pi, \pi]\text{rad}$.

C. VLA Training and Deployment

C.1. Implementation Details

LIBERO. Official publicly released checkpoints are used. No additional fine-tuning is needed.

Real World. For real-world and real-to-sim experiments, all VLA policies are fine-tuned using the official open-source implementations and their recommended training configurations. For each VLA, a single multi-task policy is trained using demonstrations from all tasks, rather than training separate policies for individual tasks. None of the pretrained VLA policies

Algorithm 1 Automatic Action Replay

Require: Recorded action sequence $\mathbf{A} = \{\mathbf{a}_t\}_{t=1}^T$, estimated initial object poses $\hat{\mathbf{P}}$, retry limit N , perturbation distribution $\mathcal{D}$

Ensure: Replayed simulation trajectory τ_{sim} and replay status

```
1: for  $i = 1$  to  $N$  do
2:   if  $i = 1$  then
3:      $\mathbf{P}^{(i)} \leftarrow \hat{\mathbf{P}}$ 
4:   else
5:      $\epsilon^{(i)} \sim \mathcal{D}$ 
6:      $\mathbf{P}^{(i)} \leftarrow \hat{\mathbf{P}} + \epsilon^{(i)}$ 
7:   end if
8:   Reset the simulator using  $\mathbf{P}^{(i)}$ 
9:   Initialize the robot from the first recorded joint state
10:   $\tau_{\text{sim}} \leftarrow \text{EXECUTE}(\mathbf{A})$ 
11:  if  $\text{SUCCESS EVALUATOR}(\tau_{\text{sim}})$  then
12:    return  $\tau_{\text{sim}}, \text{SUCCESS}$ 
13:  end if
14: end for
15: return  $\emptyset, \text{failure}$ 
```

natively supports the target action dimensionality. Therefore, the action projection layer is initialized by partially loading the pretrained weights, while the newly introduced dimensions are randomly initialized. For each VLA family, the real-world and simulation-side counterparts are initialized from the same pretrained checkpoint and use identical model-specific training recipes, update budgets, task definitions, and observation/action interfaces. Training recipes may differ across VLA families, following their respective official implementations. Unless otherwise specified, no architecture-specific modifications are introduced beyond task-dependent training hyperparameters. Specifically, the original OpenVLA autoregressively predicts the discrete token sequence of a single action at each policy query and does not natively support action chunking. To ensure a consistent action interface across all evaluated VLAs, we extend its autoregressive decoder to predict action chunks. The official repositories used in this work are listed below.

- π_0 : <https://github.com/Physical-Intelligence/openpi>
- $\pi_{0.5}$: <https://github.com/Physical-Intelligence/openpi>
- OpenVLA: <https://github.com/openvla/openvla>
- Nora-Long: <https://github.com/declare-lab/nora>
- X-VLA: <https://github.com/2toinf/X-VLA>
- SmolVLA: <https://github.com/huggingface/lerobot>

C.2. Detailed Training Parameters

The detailed training hyperparameters are shown in Tab. XI for π_0 , Tab. XII for $\pi_{0.5}$, Tab. XIII for OpenVLA, Tab. XIV for Nora-Long, Tab. XV for X-VLA, and Tab. XVI for SmolVLA.

C.3. Detailed Deployment Parameters

In each policy query, every VLA predicts a 25-step action chunk. The first five actions are executed before the policy is queried again. This receding-horizon execution protocol is used for all policies.

C.4. Computational Resources

All VLA policies are fine-tuned on NVIDIA H20 GPUs until the training objective stabilizes and no further improvement is observed. Policy inference and simulation are conducted on NVIDIA GeForce RTX 4090 GPUs.

D. VLM Preference Evaluation

D.1. Prompt Templates

The following prompt template is used for all VLM descriptions. The placeholder $\{\text{video_name}\}$ denotes the manipulation task corresponding to the input video. The actual input consists of the video together with its task name,

but contains no policy, model, checkpoint, or outcome information.

You are a professional video analysis and robot behavior evaluation expert.

Your job is to describe the video {video_name} with a strict focus on robot motion quality. Do NOT give a generic description. Do NOT summarize only the good parts. If there are flaws, even small ones, you must describe them explicitly.

You MUST describe the video through the following four dimensions:

1. Motion Smoothness

- Describe whether movements are steady, fluid, and continuous.
- Mention visible hesitation, jerky motions, jitter, pauses, or unnecessary adjustments.

2. Action Continuity / Temporal Coherence

- Describe whether the action sequence flows naturally from step to step.
- Mention if transitions are abrupt, repetitive, or uncoordinated.
- Identify any breaks in continuity or wasted motions.

3. Task Success / Goal Completion:

- Evaluate task success STRICTLY based on robot actions that are clearly initiated and actually completed in the video.
- Focus ONLY on what the robot demonstrably does, not on what could or should have been done.
- Describe concrete executed substeps (e.g., grasp, lift, place) and whether each substep is successfully completed.
- If any initiated substep is not completed (e.g., grasp attempted but not lifted or placed), you MUST treat it as a failure for that subtask.
- Explicitly state the final outcome using ONE of the following labels ONLY: successful, partially successful, or failed.
- Do NOT override failed or incomplete execution with an overall success judgment.
- Do NOT infer or assume any additional goals beyond the actions visibly attempted by the robot.
- Ignore untouched objects and background elements unless the robot clearly attempts and fails to interact with them.

4. Force / Effort Control:

- Describe how the robot applies force during contact, grasping, insertion, or placement.
- Indicate whether the applied force appears appropriate, excessive, insufficient, or inconsistent.
- Mention visible signs such as object wobbling, slipping, squeezing, pressing, or repeated force adjustments.
- Focus ONLY on observable effects of force application. Do NOT infer numerical force values.
- If force misapplication contributes to failure or hesitation, describe it explicitly.

Additional Rules:

- Use only what is visible in the video. Do not hallucinate.
- Always describe how the robot executes each key action.
- Use fine-grained temporal terms such as 'short pause', 'minor correction', 'smooth lift', 'slow transition', 'repeated adjustment', etc.
- Focus on the behavior of the robot, not the background unless relevant to the task.

Final Output Format:

Section 1: Motion Smoothness
Section 2: Action Continuity
Section 3: Task Success
Section 4: Force / Effort Control
Section 5: Short Overall Summary

The following prompt template is used for all VLM preference comparisons. The placeholder {desc_block} denotes the descriptions corresponding to the paired input videos.

You are an expert in robotic performance comparison. You must compare Robot A and Robot B based on action details, continuity, fluidity, and task success, and clearly determine which robot performs better.

{desc_block}

Your goal is to decide which video is better based ONLY on these criteria:

1. Motion Smoothness
2. Continuity / Fluidity
3. Task Completion
4. Force / Effort Control

Rules:

- Use ONLY the information explicitly stated in the descriptions.
- Do NOT hallucinate or infer unstated details.
- Smooth, continuous, steady actions → positive evidence.

- Jerky, hesitant, or multiple unnecessary adjustments → negative evidence.

Evaluation Procedure (you MUST follow internally, but DO NOT output any analysis):

1. Extract evidence for each video internally.
2. Score each dimension internally.
3. Internally decide which robot performs better.

Final Output Rule:

- Output ONLY one uppercase letter: "A" or "B".
- Do NOT output explanations, reasoning, analysis, or any other text.

D.2. Detailed VLM Parameters

The detailed VLM parameters for the generation of video descriptions and preference evaluation are summarized in Tab. VII.

TABLE VII
VLM INFERENCE CONFIGURATIONS.

Parameter	Value
Sampling FPS	10
Temperature	0
Max output tokens	1024
Video input	Full video
Generation strategy	Greedy (do_sample=False)
Overlength handling	Uniform temporal subsampling / truncation
Policy identity	Hidden
A/B assignment	Randomized and balanced per policy pair
Tie handling	Not allowed; forced binary A/B output
Invalid outputs	Re-queried once / excluded and resampled

D.3. Additional Results on Preference Matrix

Fig. 11 and Fig. 12 show the win-rate matrices obtained with different VLMs at the final 2,000-comparison checkpoint for LIBERO and real-to-sim settings, respectively.

D.4. Detailed Bradley–Terry Model Parameters

The detailed parameters of the Bradley–Terry model for preference aggregation are shown in Tab. VIII.

TABLE VIII
BRADLEY–TERRY MODEL FITTING AND BOOTSTRAP CONFIGURATIONS.

Parameter	Value
Optimization algorithm	MM
Maximum iterations	1000
Convergence tolerance	10^{-8}
Numerical stability constant	10^{-12}
Strength constraint	$\sum_i b_i = 1$
Latent score	$\theta_i = \log b_i$
Reported score	$s_i = 100b_i$
Bootstrap replicates	1000
Confidence level	95%
Bootstrap unit	Binomial resampling within each unordered policy pair
Comparison count	Preserved for every policy pair
Confidence interval	95% percentile interval on s_i

D.5. Additional Results on Bradley–Terry Convergence Curves

Detailed convergence curves on normalized Bradley–Terry model scores for all VLAs are shown in Fig. 13 for LIBERO and Fig. 14 for real-to-sim.

D.6. Additional Results on Bradley–Terry Scores

Detailed normalized Bradley–Terry model scores for all VLAs are shown in Tab. IX for LIBERO and in Tab. X for real-to-sim.

TABLE IX
BRADLEY–TERRY SCORES (%) PRODUCED BY DIFFERENT VLM JUDGES ON LIBERO.

VLM	π_0	$\pi_{0.5}$	OpenVLA	X-VLA	NORA	SmolVLA
Qwen3-VL-4B	20.32	20.53	11.62	17.89	15.09	14.56
Qwen3-VL-8B	20.47	20.65	11.57	18.13	14.79	14.39
Qwen3-VL-32B	20.58	20.76	11.48	18.37	14.68	14.13
Gemma-3-4B	20.35	20.74	11.65	17.55	15.08	14.63
Gemma-3-12B	20.54	20.70	11.30	18.39	14.83	14.24
Gemma-3-27B	20.20	20.52	11.58	18.24	14.94	14.52
LLaVA-OV-1.5-4B	20.62	20.80	11.34	18.03	14.79	14.43
LLaVA-OV-1.5-8B	20.88	21.09	11.35	17.72	14.62	14.34

TABLE X
BRADLEY–TERRY SCORES (%) PRODUCED BY DIFFERENT VLM JUDGES ON REAL-TO-SIM.

VLM	π_0	$\pi_{0.5}$	OpenVLA	X-VLA	NORA	SmolVLA
Qwen3-VL-4B	24.03	34.42	9.76	25.52	6.20	0.07
Qwen3-VL-8B	24.69	31.70	9.12	29.24	5.23	0.03
Qwen3-VL-32B	24.62	28.02	8.19	34.54	4.63	0.00
Gemma-3-4B	25.37	34.50	7.89	30.00	2.13	0.11
Gemma-3-12B	24.86	30.05	7.93	33.77	3.32	0.07
Gemma-3-27B	24.43	31.28	9.11	28.64	6.53	0.00
LLaVA-OV-1.5-4B	23.22	28.36	8.70	30.50	9.12	0.10
LLaVA-OV-1.5-8B	24.68	31.19	9.35	28.28	6.24	0.27

D.7. Human Annotation Protocol

For each setting, the same three annotators independently evaluate 200 task-stratified video pairs. The identities of the policies are hidden and the A/B order is randomized independently. Annotators receive the task instruction and the same videos used for VLM evaluation and select the video with higher execution quality according to the same criteria in Sec. III-C. The majority vote defines the human reference preference. The annotation set contains 66 success–success, 67 failure–failure, and 67 success–failure pairs in both LIBERO and real-to-sim settings. The unanimous agreement among the annotators is 78.5% for LIBERO and 80.0% for real-to-sim. The pairs are distributed uniformly across the 40 LIBERO tasks and as evenly as possible across the seven real-to-sim tasks. The annotator instructions are as follows. The placeholder `{video_block}` denotes the input videos.

You are an expert in robotic performance comparison. You must compare Robot A and Robot B based on action details, continuity, fluidity, and task success, and clearly determine which robot performs better.

`{video_block}`
Your goal is to decide which video is better based ONLY on these criteria:

1. Motion Smoothness
2. Continuity / Fluidity
3. Task Completion
4. Force / Effort Control

Rules:

- Use ONLY the information explicitly stated in the videos.
- Do NOT hallucinate or infer unstated details.
- Smooth, continuous, steady actions → positive evidence.
- Jerky, hesitant, or multiple unnecessary adjustments → negative evidence.

Final Output Rule:

- Output ONLY one uppercase letter: "A" or "B".
- Do NOT output explanations, reasoning, analysis, or any other text.

D.8. Order-Swap Evaluation

For each VLM evaluator, we reuse the same video descriptions and repeat preference evaluation after reversing the A/B presentation order for each task-stratified real-to-sim video pair. A comparison is considered consistent when both queries select the same underlying video after mapping the outputs back to video identity. The mean order-swap consistency across the eight VLM evaluators is 91.5%. The swap test is used only as a position-bias audit and does not alter the main preference matrix.

D.9. Pairwise Sampling Details

For each unordered policy pair, we evaluate checkpoints from 500 to 2,000 comparisons per unordered policy pair in increments of 100. Comparisons are distributed as evenly as possible across tasks, with counts differing by at most one when the budget is not divisible by the number of tasks. The two videos in each comparison share the same task and initial configuration. Task-level win counts are pooled into one preference matrix before BT fitting. The video pairs are sampled with replacement.

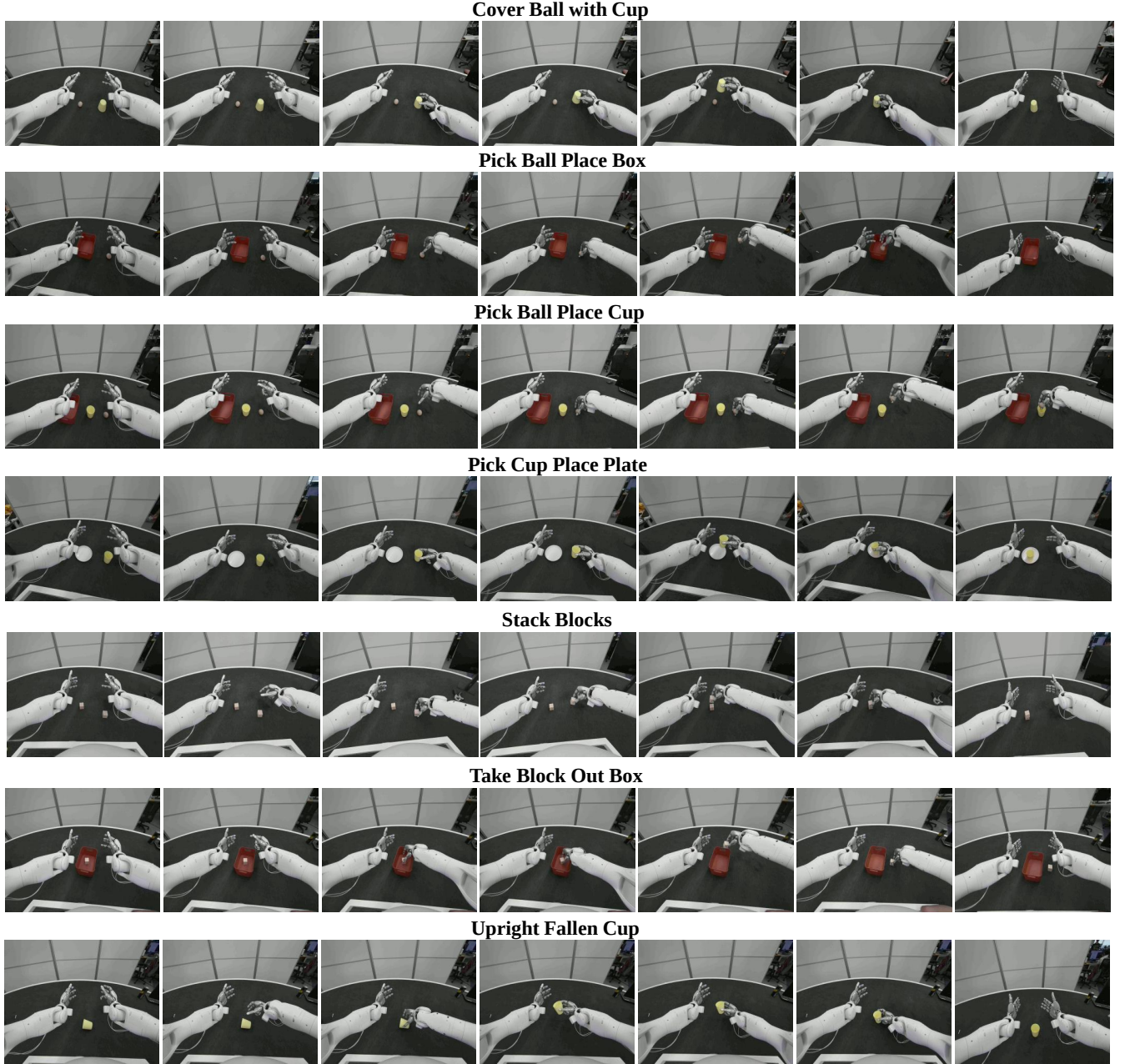
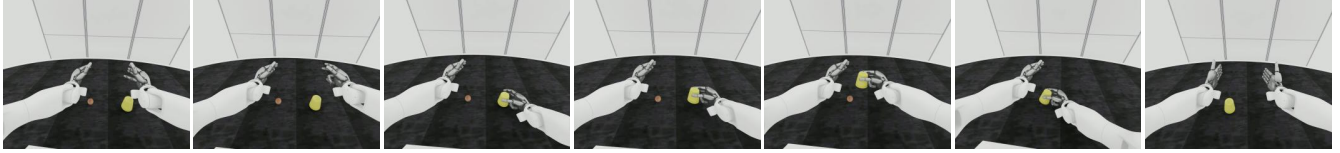


Fig. 9. Seven real-world manipulation tasks. Each row shows seven frames from left to right.

Cover Ball with Cup



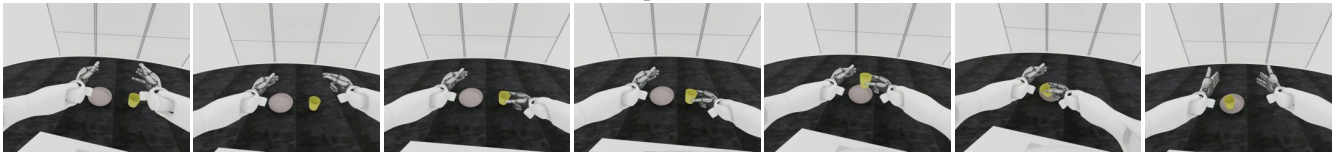
Pick Ball Place Box



Pick Ball Place Cup



Pick Cup Place Plate



Stack Blocks



Take Block Out Box



Upright Fallen Cup



Fig. 10. Seven simulated manipulation tasks. Each row shows seven frames from left to right.

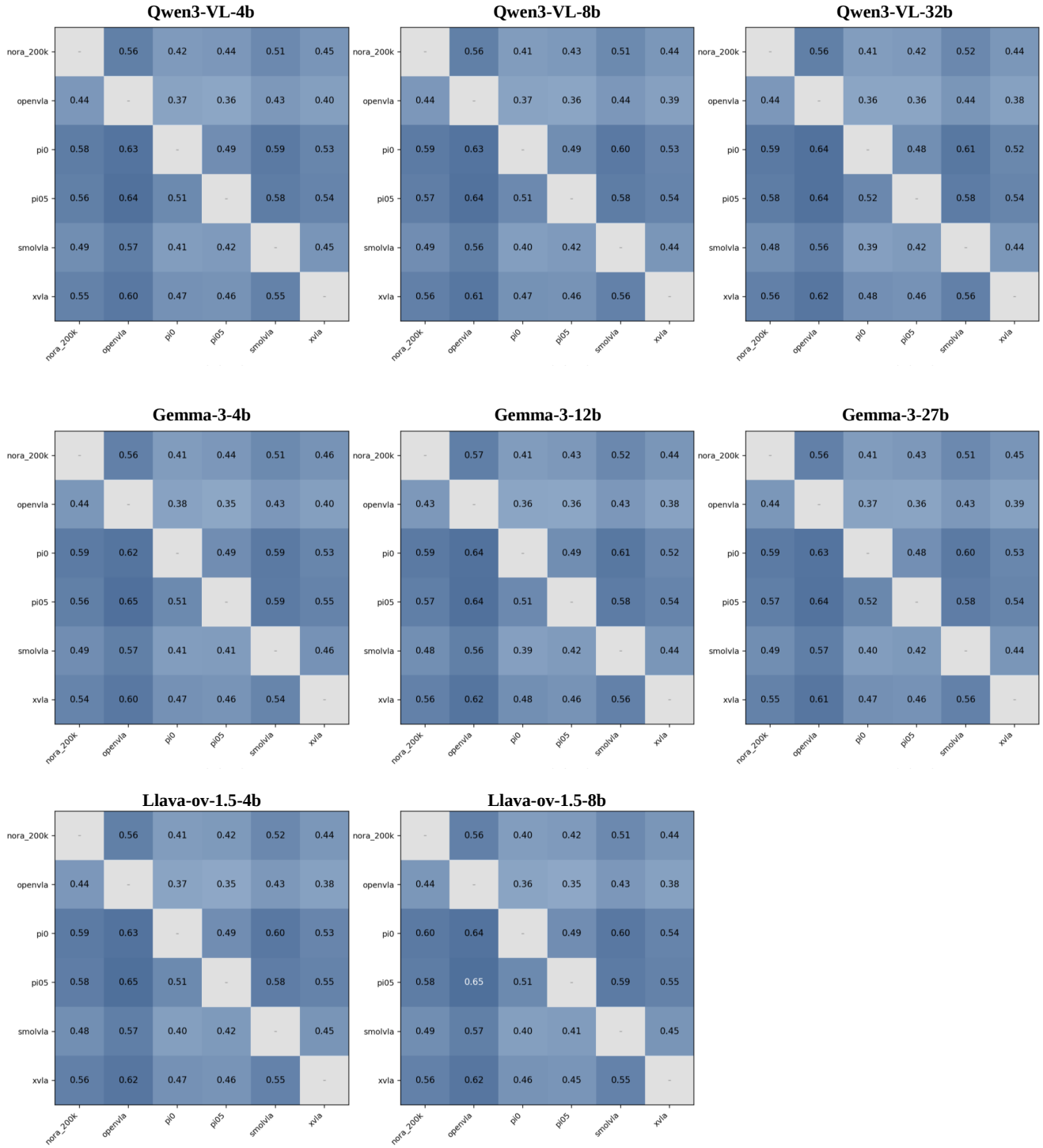


Fig. 11. LIBERO win-rate matrices obtained with different VLMs at the 2,000-comparison checkpoint.



Fig. 12. Real-to-Sim win-rate matrices obtained with different VLMs at the 2,000-comparison checkpoint.

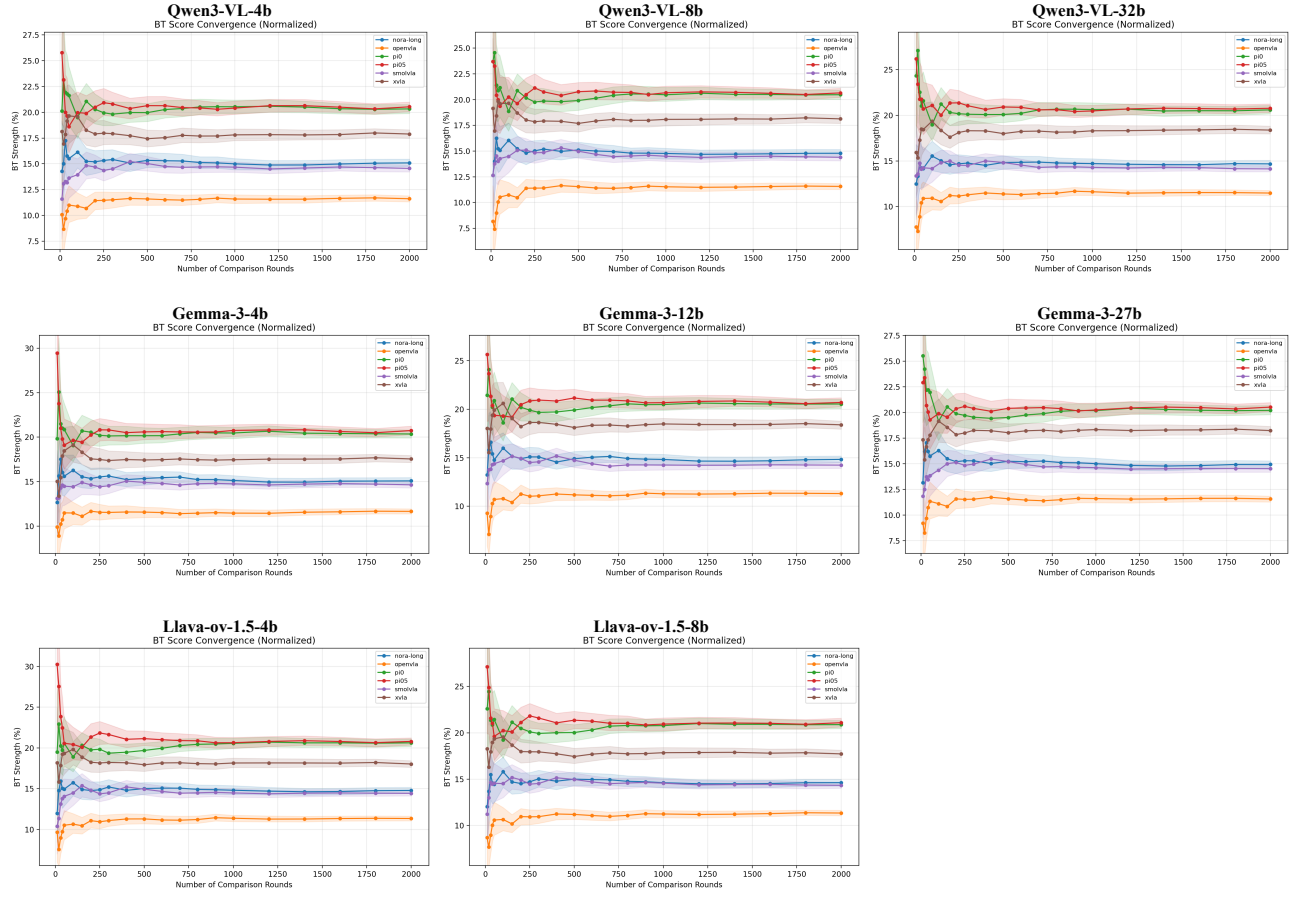


Fig. 13. Convergence curves of normalized Bradley-Terry scores for the six VLA policies under different VLM evaluators on LIBERO.

TABLE XI
TRAINING HYPERPARAMETERS FOR π_0 .

Hyperparameter	Value
Fine-tuning method	Full fine-tuning (no freeze, no LoRA)
Action representation	Flow matching, delta actions ($a - s$)
Action dimension	65
Action horizon	25
Max token length	48
Cameras / image keys	4 (head L/R stereo, left/right wrist)
Image resolution	224×224
Global batch size	32
Training steps	300,000
Optimizer	AdamW ($\beta_1 = 0.9, \beta_2 = 0.95, \epsilon = 10^{-8}$)
Weight decay	1×10^{-10}
Gradient clip (global norm)	1.0
LR schedule	Cosine decay
Peak / end LR	$2.5 \times 10^{-5} / 2.5 \times 10^{-6}$
Warmup steps	2,000
EMA decay	0.9999

TABLE XII
TRAINING HYPERPARAMETERS FOR $\pi_{0.5}$.

Hyperparameter	Value
Fine-tuning method	Full fine-tuning (no freeze, no LoRA)
Action representation	Flow matching, delta actions ($a - s$)
State input	Discrete language tokens
Action dimension	65
Action horizon	25
Max token length	300
Cameras / image keys	4 (head L/R stereo, left/right wrist)
Image resolution	224×224
Global batch size	32
Training steps	300,000
Optimizer	AdamW ($\beta_1 = 0.9, \beta_2 = 0.95, \epsilon = 10^{-8}$)
Weight decay	1×10^{-10}
Gradient clip (global norm)	1.0
LR schedule	Cosine decay
Peak / end LR	$2.5 \times 10^{-5} / 2.5 \times 10^{-6}$
Warmup steps	2,000
EMA decay	0.9999

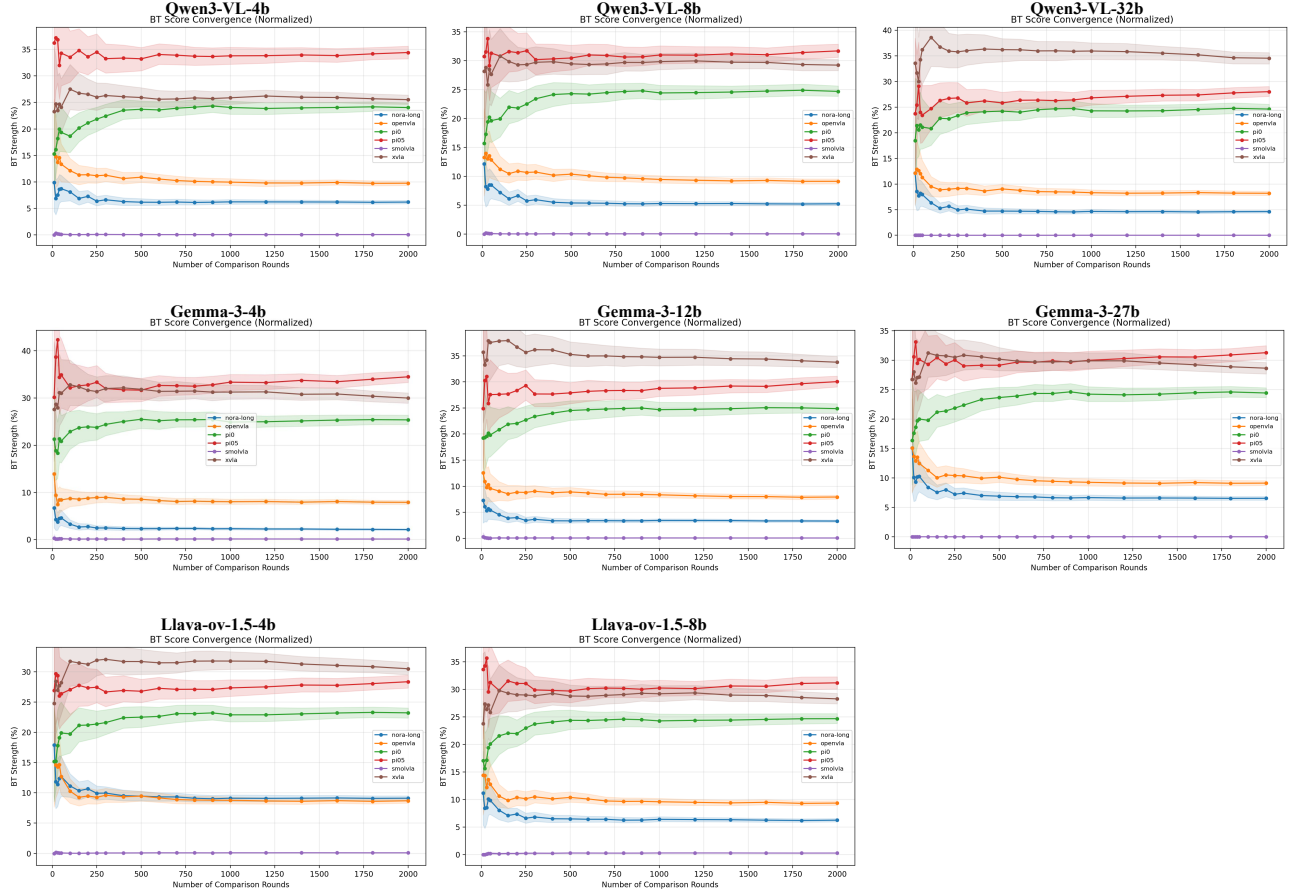


Fig. 14. Convergence curves of normalized Bradley–Terry scores for the six VLA policies under different VLM evaluators in the real-to-sim setting.

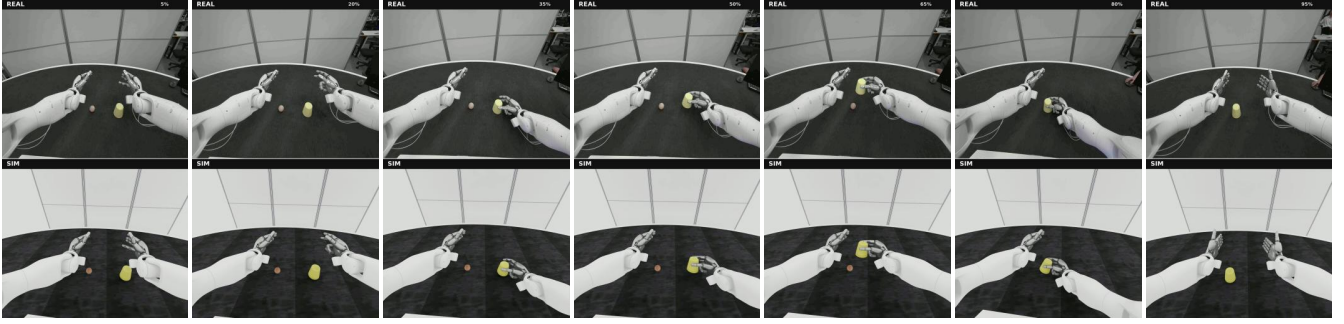
TABLE XIII
TRAINING HYPERPARAMETERS FOR OPENVLA.

Hyperparameter	Value
Fine-tuning method	LoRA (rank 32, $\alpha=16$, dropout 0.0)
LoRA target	all-linear (Gaussian init)
Action representation	Discrete tokens (256 bins/dim)
Action dim	65
Action chunk	25 (current + 24 future)
Action normalization	q01/q99 bounds $\rightarrow [-1, 1]$
Cameras / image keys	4 (head L/R stereo, left+right wrist)
Image resolution	224×224 (no aug)
Per-device / global batch	4 / 32
Grad accumulation	1
Training steps	100,000
Optimizer	AdamW
Learning rate	5×10^{-4} (constant, no warmup)
Gradient clip	None
Shuffle buffer	100,000

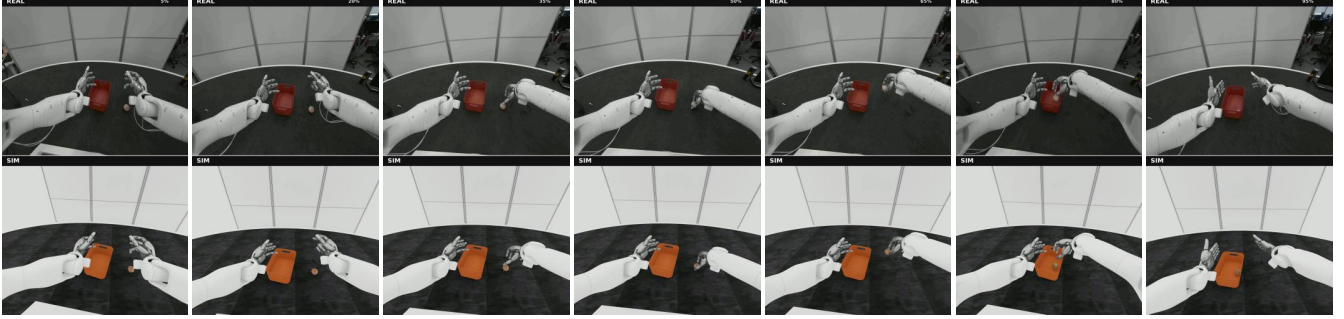
TABLE XIV
TRAINING HYPERPARAMETERS FOR NORA-LONG.

Hyperparameter	Value
Fine-tuning method	Full fine-tuning (no LoRA, no quantization)
Action representation	FAST+ tokenizer
Action dim	65
Action horizon	25 (current + 24 future)
Action normalization	q01/q99 bounds $\rightarrow [-1, 1]$
Cameras / image keys	4 (head L/R stereo, left+right wrist)
Image resolution	224×224
Per-device / global batch	8 / 64
Grad accumulation	1
Training steps	200,000
Optimizer	AdamW ($\beta_1=0.9$, $\beta_2=0.95$, $\epsilon=10^{-8}$)
Weight decay	1×10^{-8}
Gradient clip	1.0
LR schedule	Cosine
Learning rate	5×10^{-5}
Warmup steps	1,000
Shuffle buffer	25,000

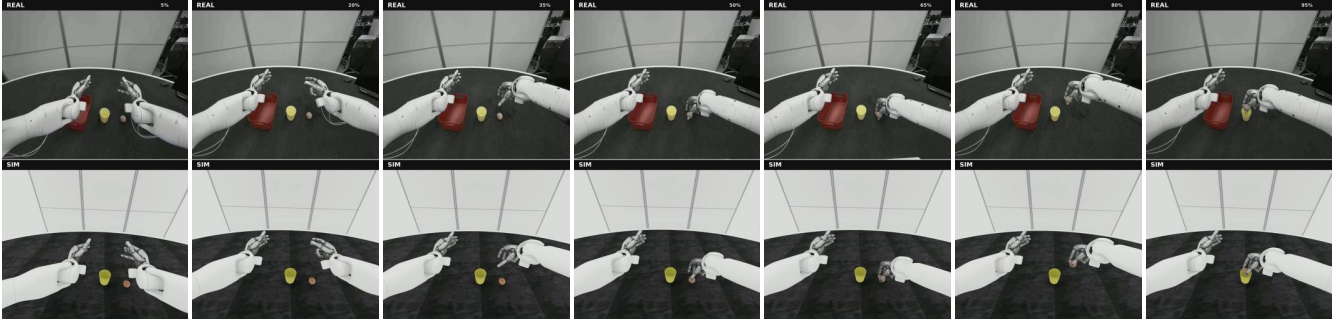
Cover Ball with Cup



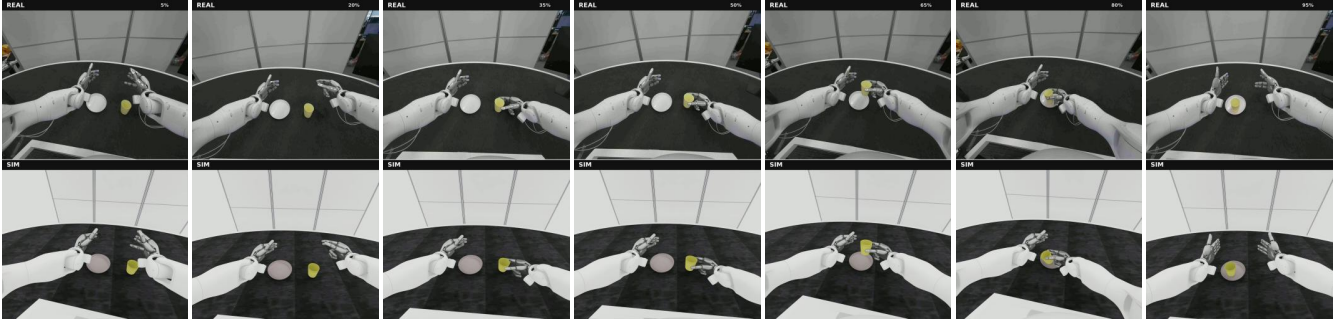
Pick Ball Place in Box



Pick Ball Place in Cup



Pick Cup Place on Plate



Stack Blocks

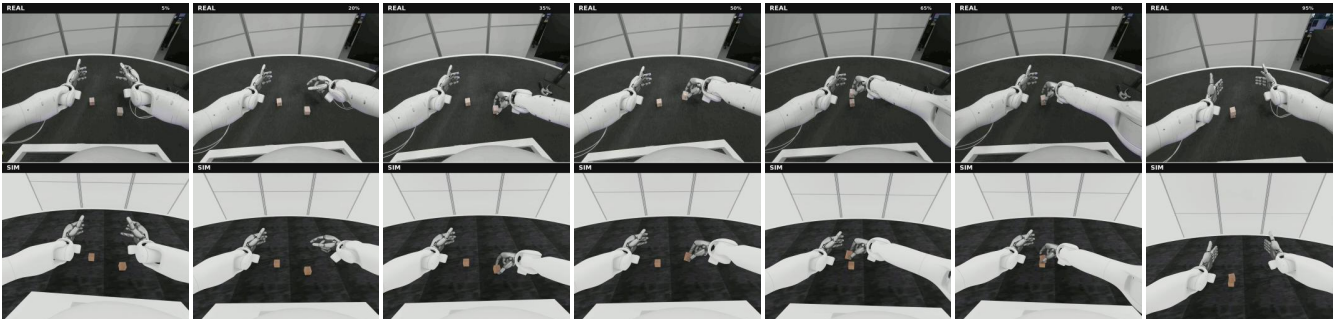


Fig. 15. Examples of real-to-sim trajectory correspondence. For each task, the top row shows the real-world trajectory and the bottom row shows its simulated action-replay counterpart. Each row contains seven frames ordered from left to right.

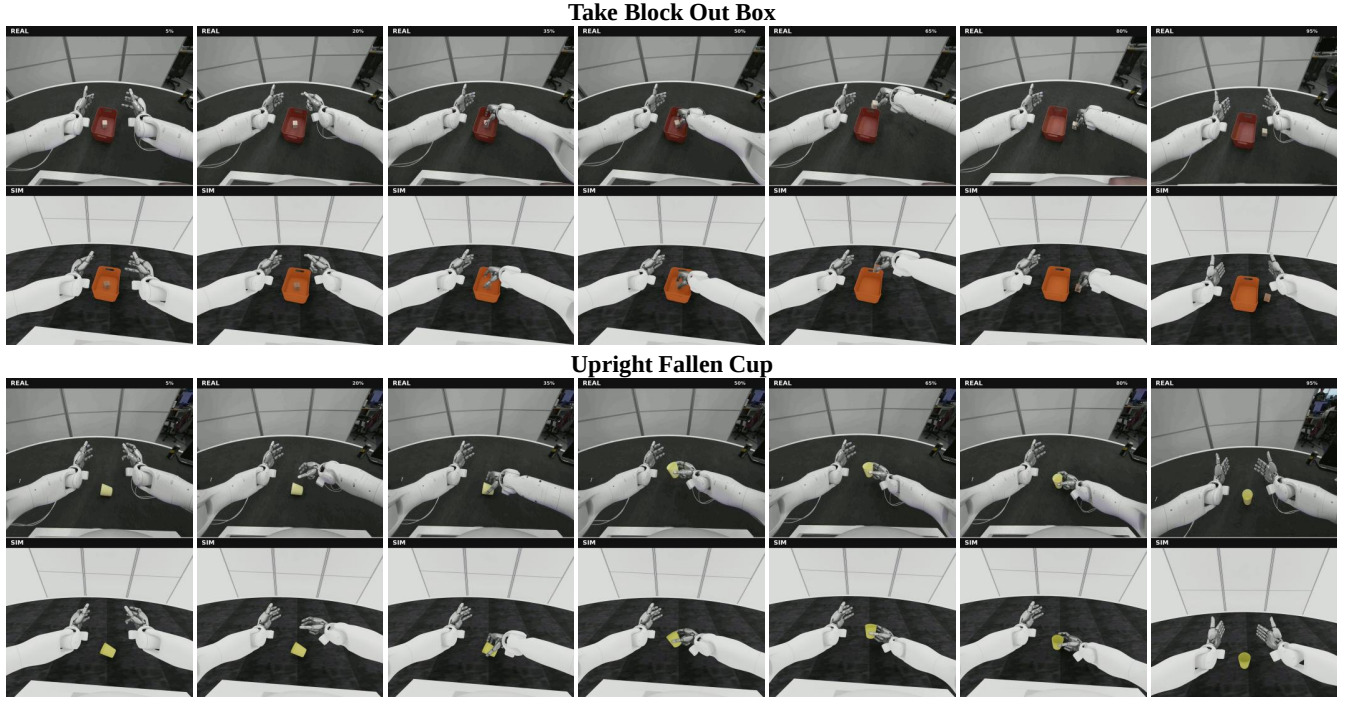


Fig. 15. Examples of real-to-sim trajectory correspondence (continued).

TABLE XV
TRAINING HYPERPARAMETERS FOR X-VLA.

Hyperparameter	Value
Soft prompt	30 domains, len 32; domain_id = 20
Action representation	Flow matching (10 denoise steps at inference)
Action dim	65
Action chunk	25
Cameras / image keys	4 (head L/R stereo, left+right wrist)
Per-device / global batch	16 / 128
Grad accumulation	1
Training steps	200,000
Optimizer	AdamW ($\beta_1=0.9$, $\beta_2=0.95$)
Weight decay	0.0
Gradient clip	1.0
Learning rate	5×10^{-5} (learning_coef 0.1: VLM/prompt 5×10^{-6})
Freeze steps	1,000 (only heads+prompts trained first)
Warmup steps	2,000 (linear); constant afterwards

TABLE XVI
TRAINING HYPERPARAMETERS FOR SMOLVLA.

Hyperparameter	Value
Fine-tuning method	Full fine-tuning (vision encoder unfrozen)
Action representation	Flow matching (10 denoise steps)
Chunk size	25
Obs steps	1
Action dim	65
Cameras / image keys	4 (head L/R stereo, left+right wrist)
Image resolution	224×224
VLM layers / attention	16, cross-attn; expert width $\times 0.75$
Tokenizer max length	48
Per-device / global batch	16 / 128
Grad accumulation	1
Training steps	200,000
Optimizer	AdamW ($\beta_1=0.9$, $\beta_2=0.95$, $\epsilon=10^{-8}$)
Weight decay	1×10^{-10}
Gradient clip (norm)	10
LR schedule	Cosine decay with warmup
Peak / end LR	5×10^{-5} / 2.5×10^{-6}
Warmup / decay steps	1,000 / 200,000